

Laboratorio: Sistema que aprende a fallar

Stefania Collazos

Fase 1 - Observar

Explicación: Apagué el servicio de backend con `docker-compose stop backend` y realicé varias peticiones al gateway en `/mascotas`. Observé que el sistema no tenía ninguna protección: cada petición intentaba conectarse al backend, esperaba 2 segundos hasta que vencía el timeout, contaba un fallo, y al llegar al tercer fallo abría el circuito. Una vez abierto, respondía con error 503 instantáneo pero nunca intentaba recuperarse solo. El servicio quedaba bloqueado para siempre hasta reiniciar el gateway.

Evidencia

Sistema funcionando

```
< > ↻ 🌐 127.0.0.1:5000/mascotas
Impresión con formato estilístico ■
{"mascotas":[[1,"Firulais","perro",null],[2,"Akyra","Perro",null],[3,"Firulais","perro",1]]}
```

Apagando servicio backend

```
C:\Users\USUARIO\Documents\pet_shop>docker-compose stop backend
[+] stop 1/1
✔ Container pet_shop-backend-1 Stopped
```

Petición 1

```
< > ↻ 🌐 127.0.0.1:5000/mascotas
Impresión con formato estilístico ■
{"error":"Servicio no disponible"}
```

5 Peticiones después

```
< > ↻ 🌐 127.0.0.1:5000/mascotas
Impresión con formato estilístico ■
{"error":"Servicio temporalmente bloqueado"}
```

LOGS

```
Símbolo del sistema
C:\Users\USUARIO\Documents\pet_shop>docker logs pet_shop-gateway-1
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.20.0.5:5000
Press CTRL+C to quit
172.20.0.1 - - [06/May/2026 03:13:03] "GET /mascotas HTTP/1.1" 200 -
Fallo número 1
172.20.0.1 - - [06/May/2026 04:09:35] "GET /mascotas HTTP/1.1" 503 -
Fallo número 2
172.20.0.1 - - [06/May/2026 04:09:56] "GET /mascotas HTTP/1.1" 503 -
Fallo número 3
Círculo abierto
172.20.0.1 - - [06/May/2026 04:10:02] "GET /mascotas HTTP/1.1" 503 -
172.20.0.1 - - [06/May/2026 04:10:03] "GET /mascotas HTTP/1.1" 503 -
172.20.0.1 - - [06/May/2026 04:10:06] "GET /mascotas HTTP/1.1" 503 -
```

- ¿Qué hace el sistema actualmente?

R: Cuando el backend está apagado, el gateway intenta conectarse y falla. Cada vez que falla, suma un contador interno. Cuando llega a 3 fallos, marca el circuito como "abierto" y deja de intentarlo. Después del tercer fallo, responde directo con un error 503 sin siquiera intentar conectarse al backend.

- ¿Se protege o insiste?

R: Insiste durante los primeros 3 intentos (sin protección). Después del tercero, se protege y ya no vuelve a intentar la conexión hasta que alguien reinicie el gateway. El problema es que nunca se recupera solo: si el backend vuelve, el gateway sigue bloqueado para siempre.

Fase 2 - Aplicar

Explicación: Reorganicé la lógica del circuit breaker usando un diccionario llamado circuitos y tres funciones reutilizables: verificar_circuito, registrar_exito y registrar_fallo. Apliqué esta lógica tanto en /mascotas como en /usuarios, cada uno con su propio circuito independiente. Observé que al apagar el backend, el circuito de mascotas se abría pero /usuarios seguía respondiendo con normalidad, demostrando que los fallos de un servicio no afectan al otro.

- ¿Cada servicio debe tener su propio contador de fallos?

R: Sí. Si el backend falla no tiene sentido bloquear el servicio de usuarios. Son contadores separados porque son servicios distintos.

- ¿El circuito debe abrirse de forma independiente por servicio?

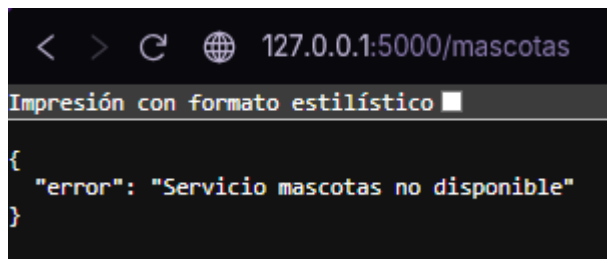
R: Sí. Si /mascotas cae, /usuarios debe seguir funcionando normal. Un circuito por servicio da más resiliencia al sistema.

- ¿Qué pasa si falla un servicio pero el otro sigue funcionando?

R: El sistema responde parcialmente: /usuarios devuelve datos normales mientras /mascotas devuelve error 503. El usuario tiene información parcial, pero el sistema no se cae entero.

Evidencia

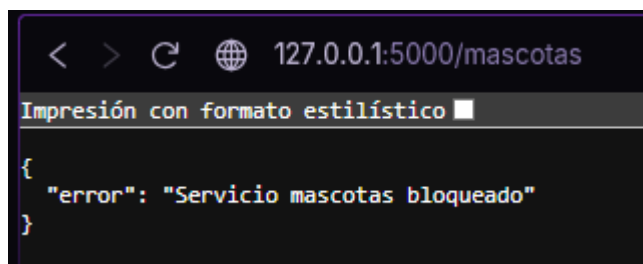
Petición 1



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5000/mascotas`. Below the address bar, there is a status bar that says "Impresión con formato estilístico". The main content area displays a JSON response:

```
{  "error": "Servicio mascotas no disponible"}
```

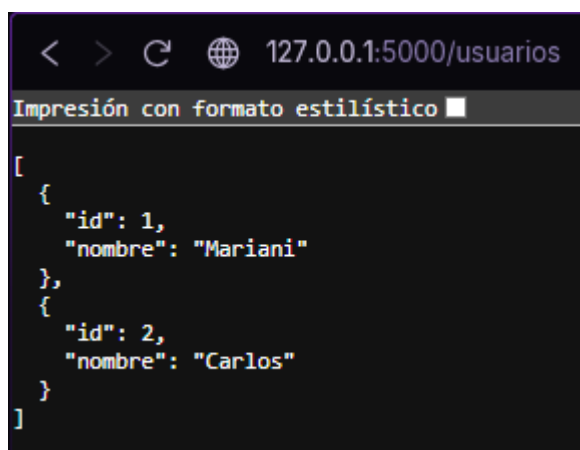
Petición 5



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5000/mascotas`. Below the address bar, there is a status bar that says "Impresión con formato estilístico". The main content area displays a JSON response:

```
{  "error": "Servicio mascotas bloqueado"}
```

Usuarios funcionando



A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5000/usuarios`. Below the address bar, there is a status bar that says "Impresión con formato estilístico". The main content area displays a JSON response:

```
[  {    "id": 1,    "nombre": "Mariani"  },  {    "id": 2,    "nombre": "Carlos"  }]
```

LOGS

```
C:\Users\USUARIO\Documents\pet_shop>docker logs pet_shop-gateway-1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.20.0.5:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 682-835-069
Fallo backend #1
172.20.0.1 - - [06/May/2026 04:44:10] "GET /mascotas HTTP/1.1" 503 -
Fallo backend #2
172.20.0.1 - - [06/May/2026 04:44:47] "GET /mascotas HTTP/1.1" 503 -
Fallo backend #3
Circuito backend ABIERTO
172.20.0.1 - - [06/May/2026 04:44:55] "GET /mascotas HTTP/1.1" 503 -
172.20.0.1 - - [06/May/2026 04:44:57] "GET /mascotas HTTP/1.1" 503 -
172.20.0.1 - - [06/May/2026 04:45:03] "GET /mascotas HTTP/1.1" 503 -
Circuito usuarios CERRADO
172.20.0.1 - - [06/May/2026 04:45:45] "GET /usuarios HTTP/1.1" 200 -
```

Fase 3 - Investigar

Explicación: Investigué el estado half-open del circuit breaker. Aprendí que es un estado intermedio entre abierto y cerrado: después de un tiempo de espera definido, el circuito deja pasar una sola petición de prueba para verificar si el servicio se recuperó. Si esa petición tiene éxito, el circuito se cierra. Si falla, vuelve a abrirse y reinicia el temporizador.

- ¿Qué significa “half-open”?

R: Es un tercer estado del Circuit Breaker, entre abierto (bloqueado) y cerrado (funcionando). Imagínalo como abrir la puerta a medias: el sistema deja pasar una sola petición de prueba para ver si el servicio caído ya se recuperó, sin abrirle paso a todo el tráfico todavía.

- ¿Cuándo se vuelve a intentar una llamada?

R: Después de que pasa un tiempo de espera definido (por ejemplo 10 segundos). Cuando ese tiempo se cumple, el circuito pasa de abierto a half-open y deja pasar la próxima petición como prueba.

- ¿Qué pasa si el servicio vuelve a fallar?

R: El circuito vuelve a abrirse completamente y reinicia el temporizador. Si la prueba tiene éxito, el circuito se cierra y el sistema vuelve a funcionar normal con todas las peticiones.

Fase 4 - Implementar

Explicación: Agregué el campo "desde" al diccionario de cada circuito para guardar el momento exacto en que se abrió. En la función verificar_circuito añadí la comparación `time.time() - c["desde"] >= Espera_segundos`: si pasaron 10 segundos, el circuito entra en half-open y deja pasar una petición de prueba. Observé en los logs el mensaje HALF-OPEN seguido de CERRADO (recuperado) cuando el servicio volvía, confirmando que la recuperación automática funcionaba correctamente.

Backend funcionando

```
< > ↻ 🌐 127.0.0.1:5000/mascotas
Impresión con formato estilístico ■
{
  "mascotas": [
    [
      1,
      "Firulaïs",
      "perro",
      null
    ],
    [
      2,
      "Akyra",
      "Perro",
      null
    ],
    [
      3,
      "Firulaïs",
      "perro",
      1
    ]
  ]
}
```

Apagando servicio backend

```
C:\Users\USUARIO\Documents\pet_shop>docker-compose stop backend
[+] stop 1/1
✓Container pet_shop-backend-1 Stopped
```

Petición 1,2 y 3

```
< > ↻ 🌐 127.0.0.1:5000/mascotas
Impresión con formato estilístico ■
{
  "error": "Servicio mascotas no disponible"
}
```

Esperamos 10 seg

Levantamos servicio backend

```
C:\Users\USUARIO\Documents\pet_shop>docker-compose start backend
[+] start 1/1
✓Container pet_shop-backend-1 Started
```

LOGS

```
C:\Users\USUARIO\Documents\pet_shop>docker logs pet_shop-gateway-1
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.20.0.5:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 677-488-718
Circuito backend CERRADO (recuperado)
172.20.0.1 - - [06/May/2026 04:59:39] "GET /mascotas HTTP/1.1" 200 -
Fallo backend #1
172.20.0.1 - - [06/May/2026 05:01:43] "GET /mascotas HTTP/1.1" 503 -
Fallo backend #2
172.20.0.1 - - [06/May/2026 05:02:07] "GET /mascotas HTTP/1.1" 503 -
Fallo backend #3
Circuito backend ABIERTO. Reintento en 10s
172.20.0.1 - - [06/May/2026 05:02:13] "GET /mascotas HTTP/1.1" 503 -
HALF-OPEN backend: probando si el servicio volvió...
Circuito backend CERRADO (recuperado)
172.20.0.1 - - [06/May/2026 05:04:12] "GET /mascotas HTTP/1.1" 200 -
```

Fase 5 - Validar

Explicación: Probé el sistema en los cuatro escenarios. Con todos los servicios activos, las peticiones respondían con normalidad y los circuitos se mantenían cerrados. Al apagar el backend, los logs mostraron los fallos acumulándose hasta que el circuito se abrió. Con el circuito abierto, las respuestas de error llegaban instantáneamente sin esperar timeout. Finalmente, al volver a levantar el backend y esperar 10 segundos, el circuito entró en half-open, la petición de prueba fue exitosa y el sistema se recuperó solo sin necesidad de reiniciar nada.

1. Servicio funcionando

Evidencia

/usuarios

```
< > ↻ 127.0.0.1:5000/usuarios
Impresión con formato estilístico
[
  {
    "id": 1,
    "nombre": "Mariani"
  },
  {
    "id": 2,
    "nombre": "Carlos"
  }
]
```

/mascotas

```
< > ↻ 127.0.0.1:5000/mascotas
Impresión con formato estilístico
{
  "mascotas": [
    [
      1,
      "Firulais",
      "perro",
      null
    ],
    [
      2,
      "Akyra",
      "Perro",
      null
    ],
    [
      3,
      "Firulais",
      "perro",
      1
    ]
  ]
}
```

2. Servicio caído

Evidencia

Apagando servicio backend

```
C:\Users\USUARIO\Documents\pet_shop>docker-compose stop backend  
[+] stop 1/1  
✓Container pet_shop-backend-1 Stopped
```

LOGS

```
00  
Circuito usuarios CERRADO (recuperado)  
172.20.0.1 - - [06/May/2026 05:09:29] "GET /usuarios HTTP/1.1" 200 -  
Circuito backend CERRADO (recuperado)  
172.20.0.1 - - [06/May/2026 05:10:25] "GET /mascotas HTTP/1.1" 200 -  
Fallo backend #1  
172.20.0.1 - - [06/May/2026 05:13:03] "GET /mascotas HTTP/1.1" 503 -  
Fallo backend #2  
172.20.0.1 - - [06/May/2026 05:13:09] "GET /mascotas HTTP/1.1" 503 -  
Fallo backend #3  
Circuito backend ABIERTO. Reintento en 10s  
172.20.0.1 - - [06/May/2026 05:13:15] "GET /mascotas HTTP/1.1" 503 -
```

3. Circuito abierto

Evidencia

```
< > ↻ 🌐 127.0.0.1:5000/mascotas  
Impresión con formato estilístico ■  
{  
  "error": "Servicio mascotas bloqueado"  
}
```

LOGS

```
Fallo backend #3  
Circuito backend ABIERTO. Reintento en 10s  
172.20.0.1 - - [06/May/2026 05:13:15] "GET /mascotas HTTP/1.1" 503 -  
HALF-OPEN backend: probando si el servicio volvió...  
Fallo backend #4  
Circuito backend ABIERTO. Reintento en 10s  
172.20.0.1 - - [06/May/2026 05:15:01] "GET /mascotas HTTP/1.1" 503 -  
172.20.0.1 - - [06/May/2026 05:15:03] "GET /mascotas HTTP/1.1" 503 -
```


4. Recuperación del servicio

Evidencia

Levantando nuevamente el servicio backend

```
C:\Users\USUARIO\Documents\pet_shop>docker-compose start backend
[+] start 1/1
✓Container pet_shop-backend-1 Started
```

```
Fallo backend #4
Circuito backend ABIERTO. Reintento en 10s
172.20.0.1 - - [06/May/2026 05:15:01] "GET /mascotas HTTP/1.1" 503 -
172.20.0.1 - - [06/May/2026 05:15:03] "GET /mascotas HTTP/1.1" 503 -
HALF-OPEN backend: probando si el servicio volvió...
Circuito backend CERRADO (recuperado)
172.20.0.1 - - [06/May/2026 05:18:46] "GET /mascotas HTTP/1.1" 200 -
```

Endpoint "Resumen"

```
< > ↻ 127.0.0.1:5000/resumen|
Impresión con formato estilístico ■
{
  "mascotas": {
    "mascotas": [
      [
        1,
        "Firulais",
        "perro",
        null
      ],
      [
        2,
        "Akyra",
        "Perro",
        null
      ],
      [
        3,
        "Firulais",
        "perro",
        1
      ]
    ]
  },
  "usuarios": [
    {
      "id": 1,
      "nombre": "Nas"
    },
    {
      "id": 2,
      "nombre": "Stef"
    }
  ]
}
```

```
< > ↻ 127.0.0.1:5000/resumen|
Impresión con formato estilístico ■
{
  "mascotas": {
    "error": "backend no disponible"
  },
  "usuarios": [
    {
      "id": 1,
      "nombre": "Nas"
    },
    {
      "id": 2,
      "nombre": "Stef"
    }
  ]
}
```

Análisis final

- ¿Qué cambió en el comportamiento del sistema?
R: Antes el gateway solo tenía un circuito para /mascotas y cuando se abría, quedaba bloqueado para siempre hasta reiniciar el servidor. Ahora cada servicio tiene su propio circuito independiente, entonces si el backend cae, /usuarios sigue funcionando normal. Además el sistema se recupera solo: después de 10 segundos entra en half-open, prueba si el servicio volvió y cierra el circuito automáticamente si funciona.
- ¿Qué decisiones tomaron en la implementación?
R: Decidí guardar el estado de cada circuito en un diccionario llamado circuitos en vez de tener variables sueltas por servicio. Eso permitió crear tres funciones reutilizables (verificar_circuito, registrar_exito, registrar_fallo) que cada endpoint usa, sin repetir la lógica. Elegí 10 segundos como tiempo de espera para el half-open y 3 fallos como límite para abrir el circuito.
- ¿Qué dificultades encontraron?
R: La parte más difícil fue implementar el half-open correctamente. Al principio no era claro cómo distinguir si el circuito estaba abierto por primera vez o si ya había pasado el tiempo de espera. La solución fue guardar el momento exacto en que se abrió el circuito ("desde": time.time()) y compararlo con el tiempo actual en cada petición.